



Professor : Romulo Beninca e-mail: romulo.beninca@ifsc.edu.br

Lista de Exercícios – Android com Java: ViewGroups e Widgets

Aluno: Jonathan Emmanuel de Oliveira Amorim

- ✓ Crie um projeto Android no Android Studio ou use uma branch de um projeto já existente.
- ✓ Utilize **LinearLayout** ou **ConstraintLayout** conforme indicado
- ✓ Use **strings.xml** e **styles.xml** para manter boas práticas de organização
- ✓ Faça commits claros com as alterações realizadas
- ✓ Suba o projeto para um repositório GitHub público, preferencialmente em uma branch chamada **layouts**

-
- 1) Explique o conceito de ViewGroup no Android, destacando seu papel na construção da interface gráfica. Em seguida, descreva o funcionamento de pelo menos dois dos principais tipos de ViewGroup (**LinearLayout** e **ConstraintLayout**), incluindo exemplos de código XML para cada um.

R:

De acordo com a documentação do android, as ViewGroups são uma visualização especial que pode conter outras visualizações chamadas de children (filhas). Ela atua principalmente como base para layouts e contêineres de visualizações (telas).

- **LinearLayout**: De acordo com a própria documentação, o LinearLayout é um layout que organiza os elementos em sequência, colocando-os todos em uma única linha ou todos em uma única coluna (vertical ou horizontal dependendo da configuração).

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    <!-- A anotação "match_parent" define que o layout ocupara toda
a largura e altura da tela (width e height) -->
    android:layout_width="match_parent"
    android:layout_height="match_parent"

    <!-- Define o padding -->
    android:paddingLeft="16dp"
    android:paddingRight="16dp"

    <!-- Define a orientação do layout -->
    android:orientation="horizontal"
</LinearLayout>
<!-- Exemplo resgatado da documentação -->
```



- **ConstraintLayout:** No **ConstraintLayout** é possível posicionar cada view em relação a outras views ou em relação a própria tela. Com ele é possível criar layouts complexos e de grande porte usando uma hierarquia simples, sem a necessidade de aninhar vários **ViewGroups**. O **ConstraintLayout** veio para reduzir o uso excessivo de layouts aninhados.

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingStart="16dp"

    <TextView
        android:id="@+id/tvExemplo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Texto Exemplo"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>

    <Button
        android:id="@+id/btnExemplo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Botão"
        app:layout_constraintStart_toEndOf="@id/tvExemplo"
        app:layout_constraintTop_toTopOf="@id/tvExemplo"
        app:layout_constraintBottom_toBottomOf="@id/tvExemplo"

</androidx.constraintlayout.widget.ConstraintLayout>
```

Crie a representação e exemplo gráfico a punho, para exemplificar cada **ViewGroup**.

